ALB/ASG Config AWS Case Study

Created by: Shaquile Goff

Objective: Build a highly available website deployment utilizing Terraform and Infrastructure as Code. Configure Auto Scaling Groups and Application Load Balancers to increase availability in response to user demand.

This project will consist of the deployment of a basic webfront on EC2 servers using Terraform for infrastructure automation. The infrastructure will be deployed in an Auto Scaling Group and behind an Application Load Balancer to ensure high availability and fault tolerance.

Skills:

- Amazon S3
- Powershell
- Terraform
- Hashicorp Configuration Language (HCL)
- Auto Scaling Groups (ASG)
- Application Load Balancer (ALB)
- AWS IaaS
- AWS fault tolerance
- AWS high availability

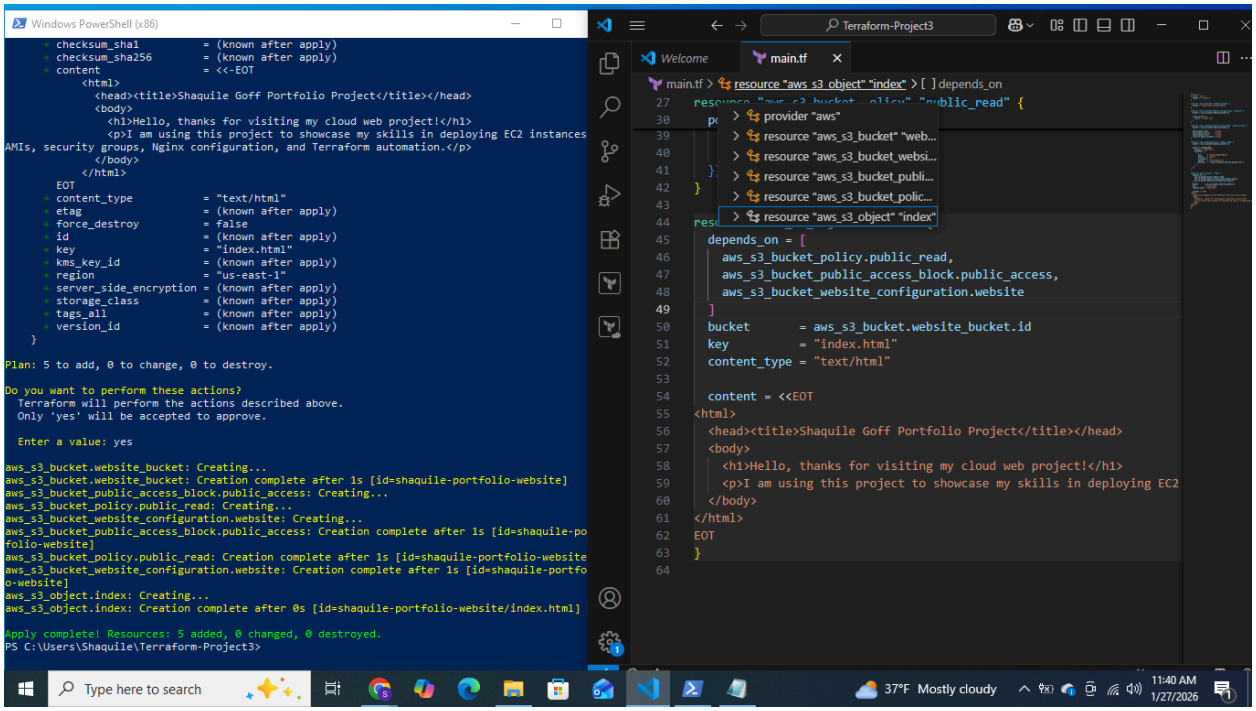
Technical Specs:

- Amazon S3 bucket
- S3 Bucket policies
- Terraform
- VS code
- HTML
- Nginx

Steps:

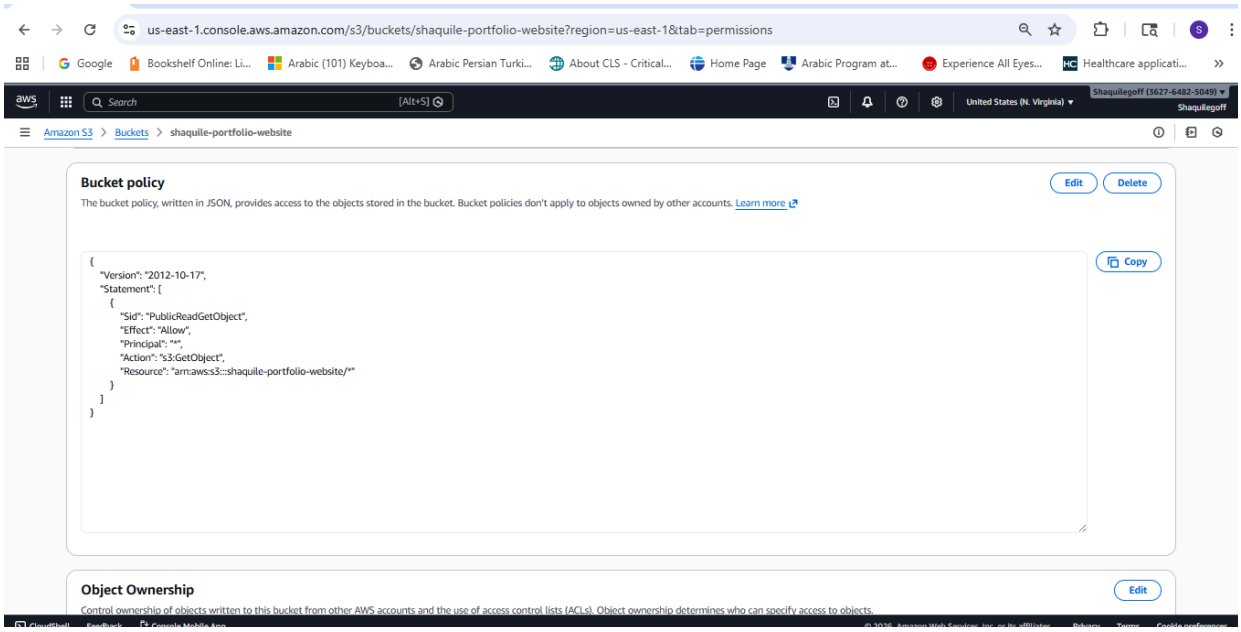
I initially defined the cloud infrastructure through terraform, which would make it easier to potentially configure cloud maintenance at a large scale like what would be encountered in a real world environment.

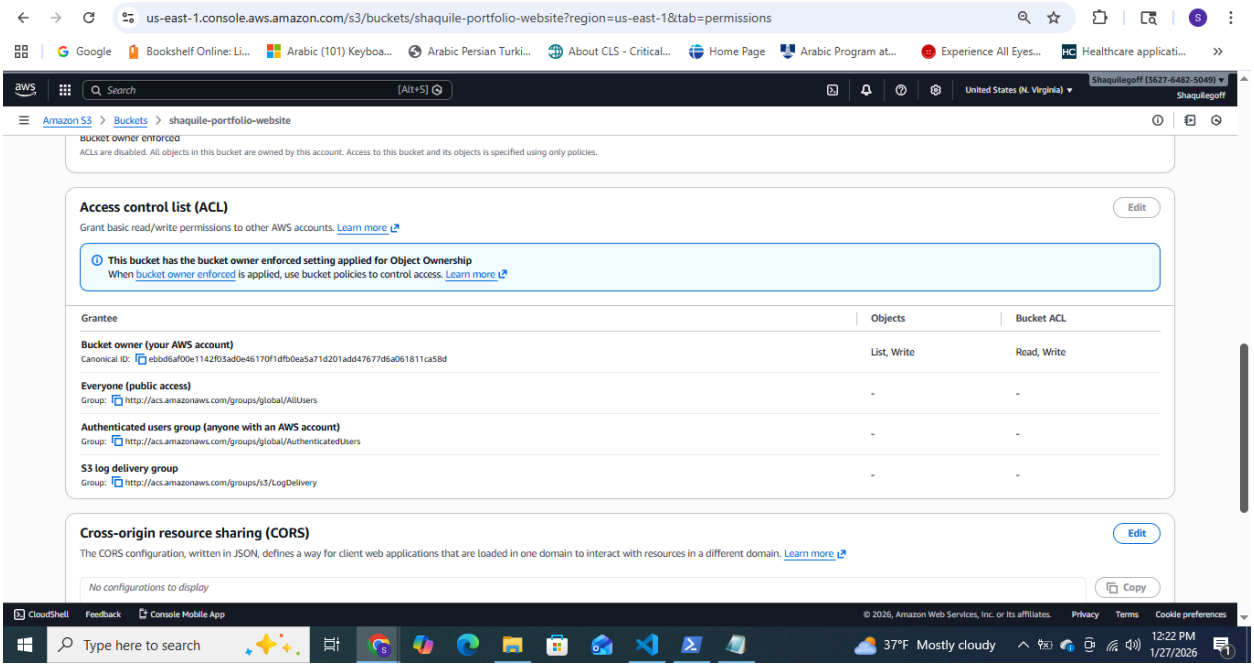
Terraform file showing creation of cloud architecture. Deployment of resources to AWS conducted via PowerShell and SSH



The next step was configuring appropriate S3 bucket policies and ACLs to ensure proper acess was allowed, while following the principal of least privilege

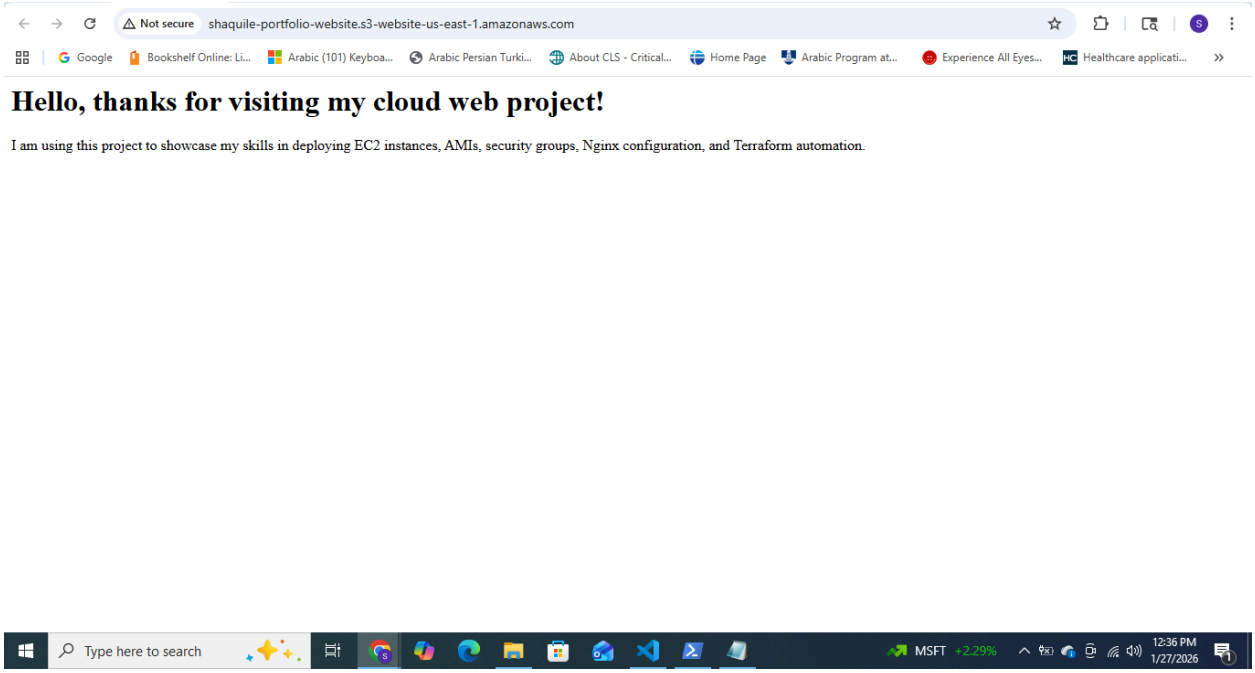
Amazon S3 bucket configurations:





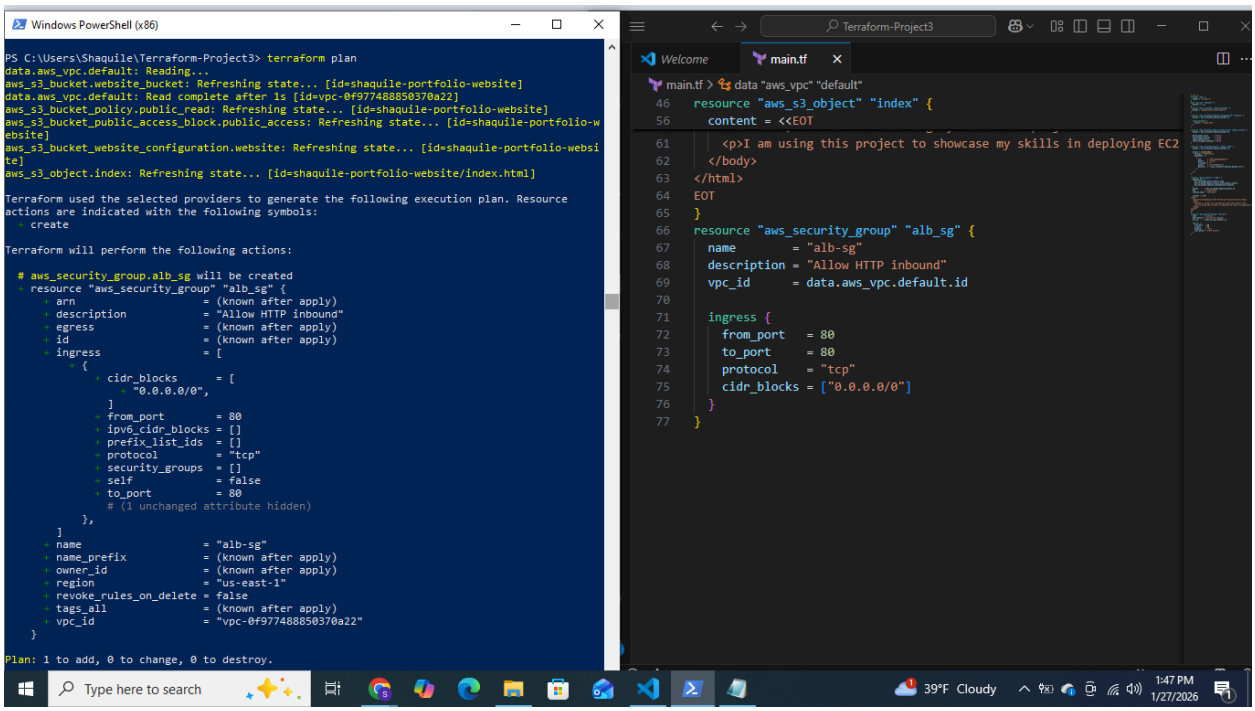
Next, I verified the web front successfully launched by connecting via the S3 endpoint.

Successful connection to the test website landing page via S3

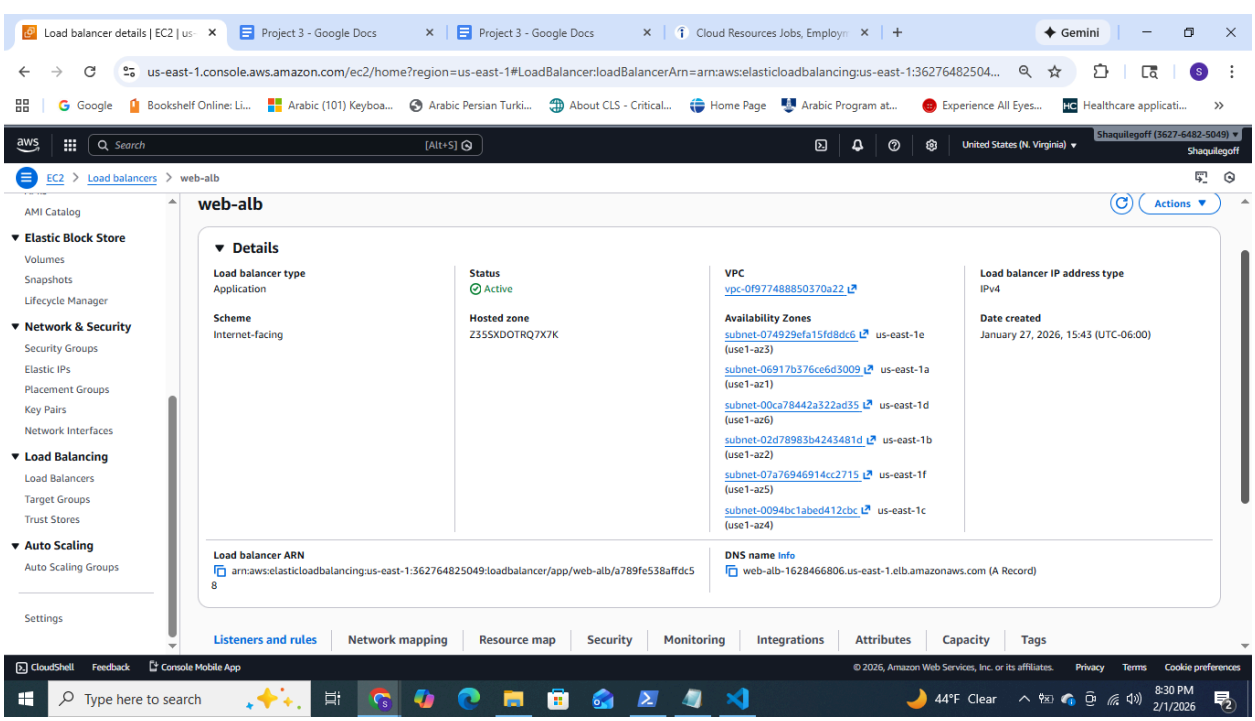


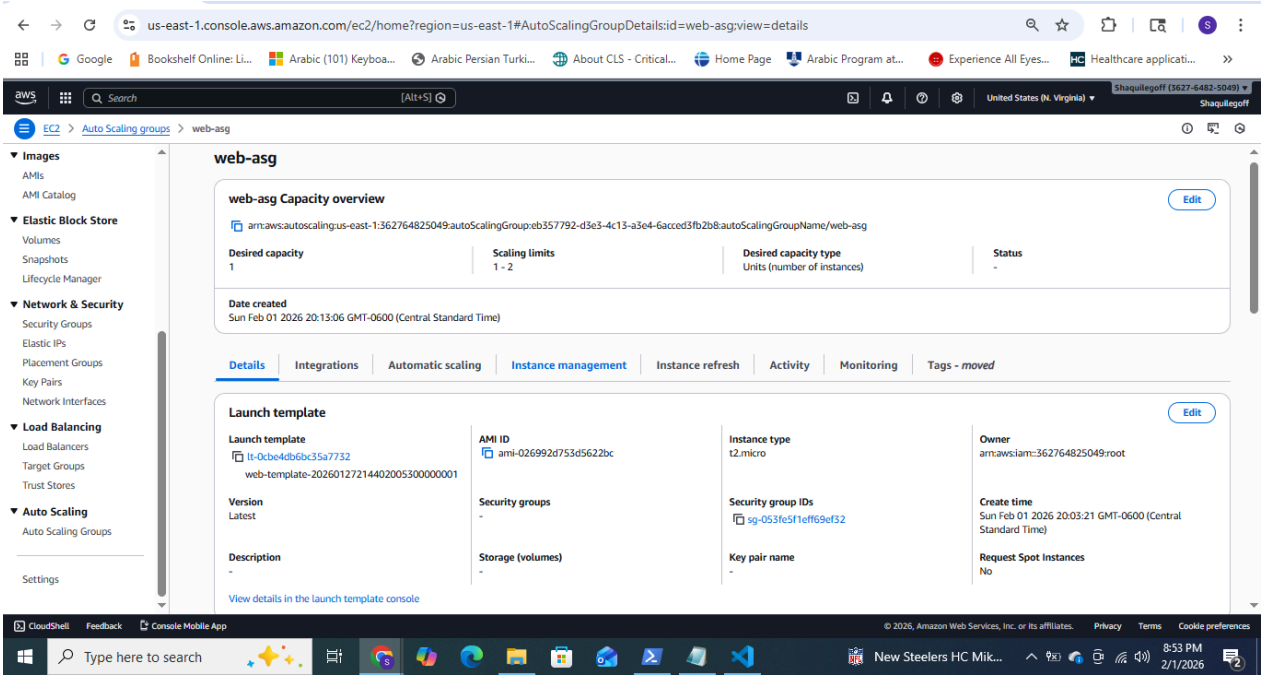
A basic website, but a successful connection. But I wanted to ensure my website would survive high network traffic, and potential server failures. I knew next I would have to put this website into an ASG and behind an ALB to complete these objectives. After enabling versioning to save the current S3 bucket, I returned to Terraform to modify the infrastructure, creating the necessary resources

Terraform execution plan showing successful creation of ALB and networking to ASG



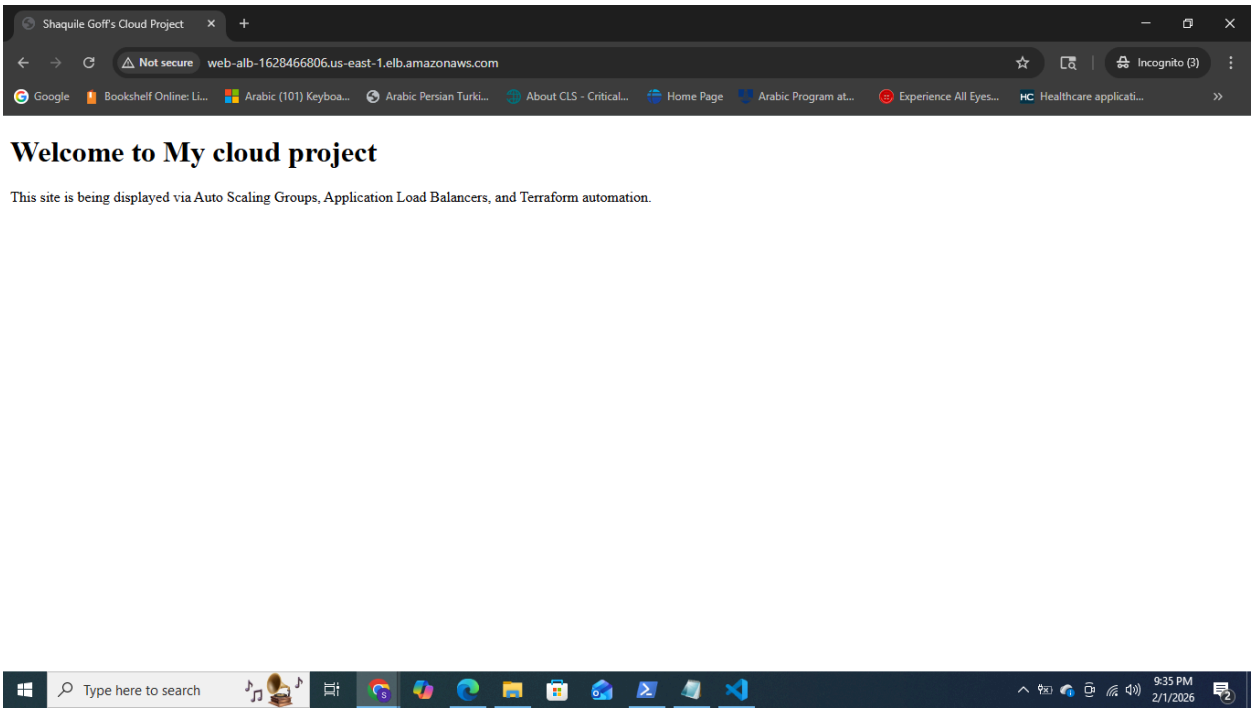
ALB and ASG successfully created and health checks validated, shown in AWS console





After validating health checks, I manually verified the accesibility of the webfront through the ALB instead of the S3 endpoint. This follows security best practices and provides a layer protecting my cloud resources from pointing directly to the internet and enables traffic to be directed based on server load and health.

Succesful connection to test page via ALB



Challenges Overcame:

- Terraform was not initially recognized in PowerShell, configured PATH environment variable. Verified functionality through PowerShell CLI after adjustments.
- Edited/Debugged issues stemming from improper Terraform syntax
- Adjusted S3 Bucket policies and account-level permissions to allow reachability over the internet.
- Utilized iterative debugging to test, refine, and fix code as needed
- Diagnosed a DNS 502 bad gateway as a block between ALB and ec2 instances
- Deployed/Redeployed ASG to implement updates
- Verified target group health to confirm functionality

Conclusion: Through this project, I demonstrated my ability to deploy a multi-tiered AWS infrastructure using Terraform for automation successfully. The deployment followed AWS guidelines for high availability and fault tolerance through the use of auto scaling groups, enabling instance deployments across several Availability Zones. Content for the application was hosted as a static object in Amazon S3 and served using Nginx.